

A Review of the Fused Lasso: Sparsity and Smoothness

Qi Er (Emma) Teng and Yiding Shan

April 15, 2026

1 Introduction

The lasso promotes sparsity but ignores relationships between ordered features, which can lead to noisy and unstable estimates in structured data. In many applications such as gene expression and mass spectroscopy, adjacent features are expected to have similar effects.

To address this limitation, Tibshirani et al. (2005) propose the fused lasso, which adds a penalty on differences between successive coefficients. This encourages both sparsity and local constancy, producing estimates that better capture structured signals.

This becomes especially important in high-dimensional settings where the number of features exceeds the sample size, and structured regularization can significantly improve model stability and interpretability.

We focus on the main methodology and intuition of the fused lasso, and omit asymptotic theory as specified in the project requirements.

2 Existing Methods and Motivation

Several methods have been proposed for regularized regression, each addressing different aspects of high-dimensional modeling:

1. **Ridge regression** applies an ℓ_2 penalty on the coefficients to shrink them toward zero. It is convex and computationally efficient, and works well when predictors are highly correlated. However, it does not produce sparse solutions, as all coefficients remain non-zero, making interpretation difficult.
2. **Lasso** applies an ℓ_1 penalty to the coefficients, encouraging sparsity and performing variable selection. It is effective even when $p > N$ and produces interpretable models. However, it treats features independently and does not account for any ordering or structural relationships among predictors, which can lead to unstable or noisy estimates in structured settings [3].
3. **Subset selection, principal components regression (PCR), and partial least squares (PLS)** provide alternative approaches for handling high-dimensional data. Subset selection directly chooses a set of predictors but is computationally expensive, while PCR and PLS construct new features from the original predictors. However, these approaches do not produce sparse, interpretable models in terms of the original variables.
4. **Fusion-based methods** penalize differences between neighboring coefficients to encourage piecewise constant profiles. These methods explicitly incorporate the ordering of predictors and can capture block structure in the coefficients. However, without an additional ℓ_1 penalty

they do not enforce sparsity. Similarly, replacing the absolute difference with a squared difference produces smoother but non-sparse and less interpretable and does not produce the piecewise constant structure that makes the fused lasso attractive [3].

The main limitation of these methods is that they address either sparsity or smoothness, but not both simultaneously. In many applications with ordered features, neighboring predictors are often correlated and expected to have similar effects. Methods like the lasso ignore this structure, while fusion-based approaches fail to eliminate irrelevant variables.

This motivates the fused lasso, which combines an ℓ_1 penalty on the coefficients with an additional penalty on differences between adjacent coefficients. This allows the model to handle both sparsity and structure in ordered data, making it better suited for ordered data [3].

Overall, the key advantage of the fused lasso is that it combines sparsity and smoothness in a single framework, which neither lasso nor fusion alone can achieve.

3 The Fused Lasso Method

The main idea of the fused lasso is to estimate regression coefficients in a way that encourages both sparsity and local constancy [3]. This is designed for problems where the predictors have a natural ordering, so that adjacent variables are expected to have related effects.

For the linear regression model

$$y_i = \sum_{j=1}^p x_{ij}\beta_j + \epsilon_i,$$

where y_i is the response for observation i , x_{ij} is the value of predictor j for observation i , β_j is the coefficient for predictor j , and ϵ_i is the error term. Here, N denotes the number of observations and p the number of predictors.

The fused lasso estimates the coefficient vector $\beta = (\beta_1, \dots, \beta_p)$ by solving the following constrained optimization problem:

$$\min_{\beta} \sum_{i=1}^N \left(y_i - \sum_{j=1}^p x_{ij}\beta_j \right)^2 \text{ subject to } \sum_{j=1}^p |\beta_j| \leq s_1 \text{ and } \sum_{j=2}^p |\beta_j - \beta_{j-1}| \leq s_2,$$

where s_1 and s_2 are nonnegative tuning parameters that control the amount of sparsity and fusion, respectively. An equivalent formulation of the fused lasso is the penalized (Lagrangian) form:

$$\min_{\beta} \sum_{i=1}^N \left(y_i - \sum_{j=1}^p x_{ij}\beta_j \right)^2 + \lambda_1 \sum_{j=1}^p |\beta_j| + \lambda_2 \sum_{j=2}^p |\beta_j - \beta_{j-1}|,$$

where λ_1 and λ_2 are nonnegative tuning parameters that control the degree of sparsity and fusion, respectively[3]. In particular, larger values of λ_1 increase sparsity by shrinking more coefficients to zero, while larger values of λ_2 encourage stronger fusion, producing longer flat regions in the coefficient profile. The balance between these two parameters determines whether the model behaves more like a lasso, a pure fusion method, or a combination of both. Thus, the fused lasso can be viewed as combining the lasso and a total variation penalty into a single framework that enforces both sparsity and local constancy.

The first penalty is the usual lasso penalty, which shrinks some coefficients exactly to zero and performs variable selection. The second penalty penalizes changes between adjacent coefficients,

encouraging neighboring coefficients to be equal. This penalty is closely related to total variation regularization, which penalizes differences between neighboring values. It can also be interpreted as encouraging sparsity in the successive differences $\beta_j - \beta_{j-1}$, which leads to many adjacent coefficients becoming exactly equal. Thus, the fused lasso enforces sparsity in both the coefficients and their successive differences, which is the defining feature of the method.

As a result, the estimated coefficients form a sparse and piecewise constant structure, where groups of neighboring predictors share the same value. This allows the model to identify regions of predictors with similar effects, rather than selecting variables independently. This structure is especially useful in applications where the true signal is expected to change in segments rather than continuously.

This also explains why the fused lasso often performs better than the lasso in structured settings. By encouraging neighboring coefficients to be equal, it reduces variance within regions and avoids selecting isolated noisy predictors. Instead of identifying individual features, it recovers groups of features that act together, which better matches the underlying structure in applications such as gene expression or spectroscopy.

3.1 Computation

The fused lasso optimization problem is convex and can be formulated as a quadratic programming problem with sparse linear constraints. The authors solve this using an active-set algorithm (SQOPT), which is designed for large-scale sparse optimization.

The authors exploit the structure of the problem to efficiently compute solutions across different tuning parameters, using warm-start strategies and sparse optimization techniques. This allows the method to remain computationally feasible even in high-dimensional settings.

3.2 Properties and Interpretation

Intuition

The fused lasso assumes that predictors are ordered and that neighboring predictors often have similar effects. At the same time, some predictors may be irrelevant and should be excluded.

The method proceeds by fitting a regression model with two penalties: one that controls the overall magnitude of the coefficients and another that controls the differences between adjacent coefficients. For given values of the tuning parameters, the optimization problem is solved to obtain $\hat{\beta}$. In practice, different values of the tuning parameters are explored (e.g., via validation or cross-validation), and the solution that gives the best predictive performance is selected.

This produces coefficients that are both sparse and piecewise constant, making the model more interpretable and better suited to structured data.

As shown in Figure 2, the unregularized estimates are noisy, while the lasso produces sparse but irregular coefficients. The fusion method produces piecewise constant coefficients but does not enforce sparsity. The fused lasso combines both properties, yielding coefficients that are both sparse and piecewise constant.

In the special case of orthonormal design, the lasso reduces to a soft-thresholding rule applied to least squares estimates. The fused lasso extends this idea by encouraging sparsity both in the coefficients themselves and in their successive differences.

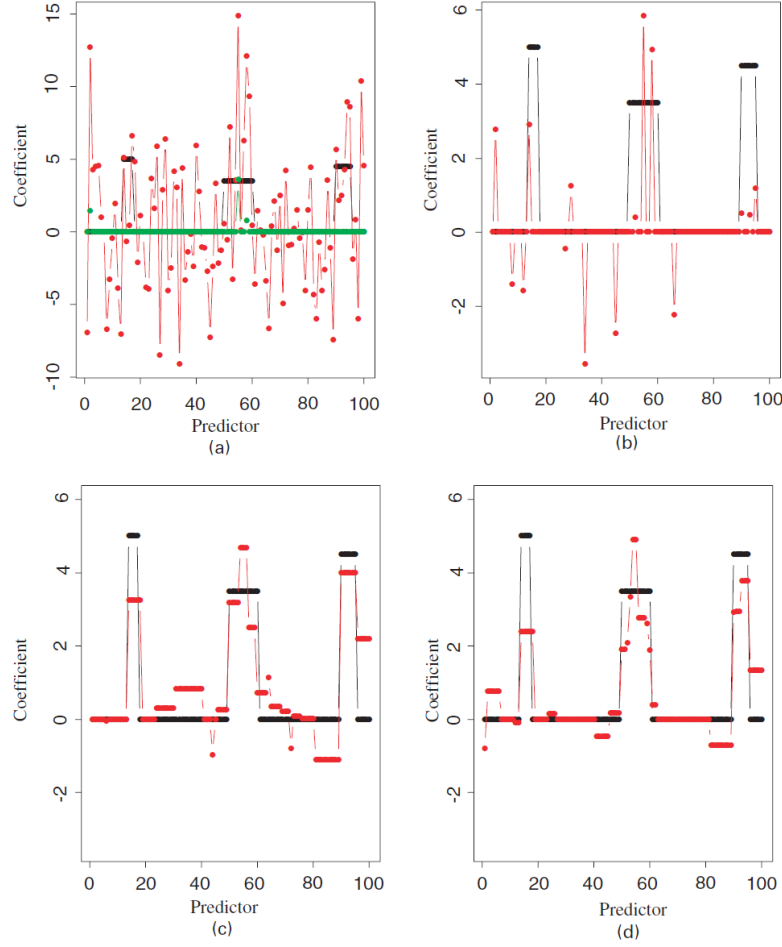


Figure 1: Simulated example adapted from [3]. (a) Unregularized regression coefficients, (b) lasso estimates showing sparsity, (c) fusion estimates showing piecewise constant structure without sparsity, (d) fused lasso estimates combining both sparsity and piecewise constant structure.

Key Properties

A key property of the fused lasso is that it produces solutions that are not only sparse, but also grouped into blocks of equal coefficients. Under mild conditions, the number of such coefficient blocks is bounded by the sample size N . This reflects a structured form of sparsity, where the model selects regions of predictors rather than individual variables.

The paper also introduces a special case called the fused lasso signal approximator, obtained when the predictor matrix is the identity matrix. In this case, the method estimates an underlying signal directly from noisy observations by shrinking the signal values toward zero and toward each other [3]. This produces fitted signals with flat regions separated by jumps, which is especially useful when the true signal is believed to be piecewise constant.

These properties suggest that the fused lasso should outperform methods like the lasso in settings where the true signal exhibits block structure. This is further illustrated in the simulation study, where the method is compared against standard approaches.

4 Simulation Study

The paper’s simulation study focused on comparing the performance between lasso and fused lasso in high dimensional regression problem. The original study used real protein mass spectroscopy data as a foundation to simulate feature matrix, and combines a synthetic piecewise-constant coefficient structure to evaluate each methods’ abilities to infer signals under high-dimensionality and limited data.

The three quantities reported in Table 3 of the paper are test error, which is the sum of squared residuals on test set; sensitive, the proportion of true nonzero coefficients that are correctly identified as nonzero; and specificity, the proportion of true zero coefficients that are correctly identified as zero.

4.1 Original Paper Setup

4.1.1 Feature Matrix

The paper constructs the feature matrix X from real prostate cancer mass spectroscopy data [1]. The raw data contains 48,538 m/z intensity measurements per patient. Following the same preprocessing used in Section 7 [3], measurements below $m/z = 2000$ are discarded to remove chemical artefacts, and the remaining sites are block-averaged in consecutive groups of 20, leaving approximately 2,181 usable features (p). The paper then takes the first 1,000 of these features and selects a random subset of 100 patients. All features are standardized to mean 0 and unit variance.

4.1.2 Coefficient Generation

The true coefficient vector β is generated to be piecewise constant and sparse. A random number of non-overlapping blocks is chosen uniformly from $\{1, \dots, 100\}$. Each block is assigned a length drawn uniformly from $\{1, \dots, 100\}$ and placed at a randomly chosen starting position among the remaining available feature indices. All features within a block receive the same value drawn from $N(0, 1)$. All other coefficients are exactly zero.

4.1.3 Response Generation

The response is generated as:

$$y = X\beta + \epsilon, \text{ where } \epsilon \sim N(0, 2.5^2).$$

The noise standard deviation of 2.5 was chosen so that the model explains approximately 50% of the variance in the test set response.

4.2 Our Replication and Adaptation

4.2.1 Dataset Availability

Our implementation follows the paper’s setup as closely as possible, but the publicly available version of the prostate cancer mass spectroscopy dataset [2] differs from the one used in the paper [1]. After applying the identical preprocessing pipeline, our dataset yields only 518 usable features rather than the paper’s over 1000 features.

The number of patients used is identical (as $N = 100$), but the class composition of our patient pool also differs slightly from the paper’s. Our dataset [2] contains a higher proportion of healthy and benign patients relative to cancer patients, whereas the paper’s dataset [1] is reversed. However,

since the simulation study uses patient spectra purely as a source of realistic correlation structure, the response y is entirely synthetic and unrelated to disease labels. Hence this class imbalance has no effect on the simulation results.

4.2.2 Feature Matrix Construction

We load patient spectral files from four groups (Benign PSA > 4 , No Evidence of Disease PSA < 1 , Prostate Cancer PSA 4-10, Prostate Cancer PSA > 10) and apply identical preprocessing pipeline: filter to $m/z \geq 2000$, block-average in groups of 20, and truncate to the minimum common length across patients. A random seed (315) is used to select 100 patients, and all 518 features are standardized to mean 0 and unit variance using sklearn’s StandardScaler applied once globally before the simulation loop begins.

4.2.3 Beta Generation Adaptations

We follow the paper’s block structure exactly:

$$\begin{aligned} &1\text{-}10 \text{ blocks, lengths } \sim \textit{Uniform}(1, \dots, 100), \\ &\text{values } \sim N(0, 1), \text{ non-overlapping placement.} \end{aligned}$$

Two additional constraints were introduced relative to the paper. First, block values are clipped to the interval $[-2, 2]$ to prevent rare extreme draws from $N(0, 1)$ from dominating the signal scale. Second, the total fraction of nonzero coefficients is capped at 10% of p (approximately 52 features). With 518 features instead of 1000, allowing up to 50% nonzero features would make the problem unrealistically dense given our smaller feature set, and would cause specificity to be undefined.

4.2.4 Signal Normalization and Response

Our most important adaptation is in response generation. The paper uses $y = X\beta + \epsilon$ with a fixed noise standard deviation of 2.5. In our replication, the block-averaged features are highly correlated with their neighbours since each feature represents a mean over 20 raw m/z sites. When a block assigns the same value to consecutive correlated features, the signal $X\beta$ can have a standard deviation that varies enormously across Monte Carlo runs. From near zero if the block value happens to be small, to very large numbers if several large correlated blocks combine. With the paper’s fixed noise at 2.5, this variability caused the signal to noise ratio to be inconsistent across runs, some runs identify true signals better, while others were intractable, hence producing wildly variable test errors. This modification does not affect the relative comparison between methods, since both lasso and fused lasso are evaluated under the same setup in each run. It mainly helps stabilize the simulations given the stronger feature correlations in our dataset, so we treat it as a practical adjustment rather than a change to the core setup.

To address this issue, we rescale β after generation so that the signal standard deviation matches the noise standard deviation:

$$\beta_{true} = \beta \cdot \frac{2.5}{std(X\beta)}.$$

The response is then generated as $y = X\beta_{true} + \epsilon$ with $\epsilon \sim N(0, 2.5^2)$. This ensures that every simulation run operates at a signal to noise ratio of approximately 1, matching the paper’s intended approximation of 50% total variance in y .

4.2.5 Lasso

We implement the lasso solution at minimum test error using Sklearn’s coordinate descent solver rather than the paper’s LAR algorithm (note that they are identical mathematically). An alpha grid of 40 logarithmically spaced values is constructed from 5% to 100% of $\alpha_{max} = \frac{\text{argmax}(X^T y)}{n}$

The lower bound of 5% was chosen to prevent the solver from exploring overly dense solutions (with more nonzero coefficients than training samples) that fit training noise and produce catastrophic test errors. Warm starts are used for computational efficiency. The coefficient vector achieving minimum test error over this grid is selected as the solution, and the corresponding λ_1 is used in the fused lasso search.

Although the paper uses the LAR algorithm to trace the full solution path, our grid search over test error plays a similar role in approximating the optimal tuning parameter, which is consistent with the selecting minimum error idea in the paper.

4.2.6 Fused Lasso Solver

The paper solves the fused lasso in its constrained form using the SQOPT quadratic programming package. We were unable to use SQOPT as it is not publicly available. We instead solve the equivalent penalized formulation using CVXPY with the CLARABEL solver:

$$\text{argmin}(\frac{1}{2n}||y - X\beta||_2^2 + \lambda_1||\beta||_1 + \lambda_2||D\beta||_1)$$

where D is the (p-1) · p first difference matrix. The $\frac{1}{2n}$ normalization factor is to scale λ_1 to Sklearn’s lasso alpha. For each run, λ_1 is fixed at the regular lasso’s best value, and a grid of 12 values is searched in the range $[\lambda_1 \cdot 10^{-2}, \lambda_1 \cdot 10^2]$ for λ_2 . The paper’s use of SQOPT allowed a more computationally efficient warm start search along a geometrically path in (s_1, s_2) space, but our grid search approach is standard practice for penalized regression with CVXPY.

4.2.7 Monte Carlo Runs

The paper used 20 Monte Carlo runs while we used 100, which provides more precise standard error estimates. All results are reproducible using the provided code and a fixed random seed (RAND_SEED = 315).

4.3 Results

Table 1: Paper Table 3 REPLICATION: Result of simulation study over 100 runs.
(Standard errors are given in parentheses)

Method	Test Error	Sensitivity	Specificity
Lasso	411.993 (11.492)	0.070 (0.004)	0.980 (0.001)
Fused lasso	387.253 (9.974)	0.400 (0.031)	0.939 (0.008)
Fused lasso (true s_1, s_2)	446.935 (13.138)	0.111 (0.010)	0.978 (0.002)

Table 2: Paper Table 3: Result of simulation study over 20 runs.
(Standard errors are given in parentheses)

Method	Test Error	Sensitivity	Specificity
Lasso	265.194 (7.957)	0.055 (0.009)	0.985 (0.003)
Fused lasso	256.117 (7.450)	0.478 (0.082)	0.693 (0.072)
Fused lasso (true s_1, s_2)	261.380 (8.724)	0.446 (0.045)	0.832 (0.018)

4.4 Discussion

4.4.1 Qualitative Agreement

Our results replicate the paper’s central finding, the fused lasso has substantially higher sensitivity than the standard lasso (0.400 vs. 0.070 in our results, 0.478 vs. 0.055 in the paper). This confirms that accounting the ordering of features and encouraging piecewise-constant solutions allows the fused lasso to recover a much larger fraction of true signal regions.

4.4.2 Quantitative Differences

Several quantitative differences exist between our results and the paper’s, which are all expected differences from the adaptations mentioned in previous section.

4.4.3 Test Errors

Test errors are higher in our replication (about 412 for lasso vs. paper’s ~ 265). This is primarily a consequence of having lower count of features (518) comparing to the paper’s implementation. With fewer features, the feature matrix, all methods have access to fewer measurements of the underlying signal, making it harder to distinguish true signal features from noise. This directly results in higher test errors on average.

4.4.4 Fused Lasso Specificity

Our fused lasso achieves higher specificity (0.939) than the paper’s (0.693), so it produces fewer false positives at the cost of lower sensitivity (0.400 vs 0.478). The difference is related to our signal normalization producing smaller absolute coefficient values relative to the penalty strength, which caused the solution to have sparser estimates.

4.5 Replication Model Performance: Coefficient Profile

Figure 2 displays the estimated coefficient profiles from a representative simulation run, plotting each method’s estimates against the true m/z axis. The yellow shaded region indicates the true nonzero block where 100 consecutive m/z features all sharing the same coefficient value of approximately -0.045.

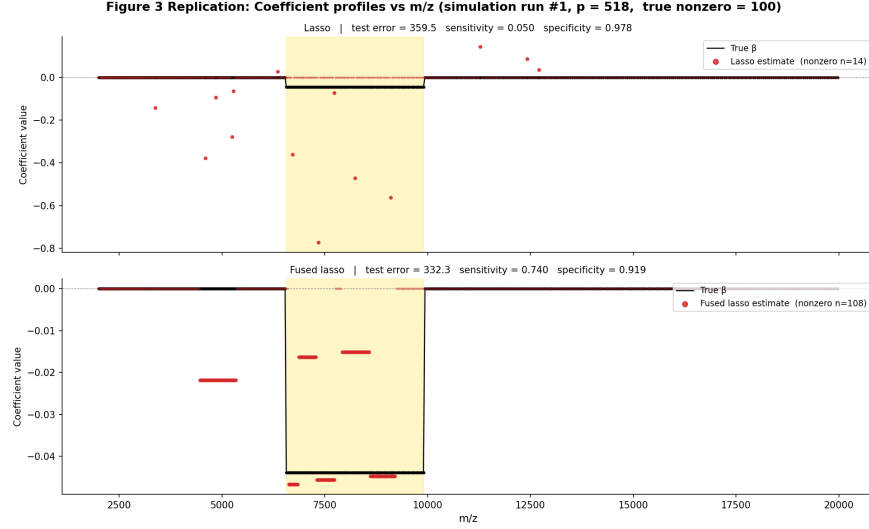


Figure 2: Replicated models' coefficient profile plot. (top) lasso estimates showing sparsity, (bottom) fused lasso estimates combining both sparsity and piecewise constant structure.

References

- [1] Davis JW Ward MD Clements MA Cazares LH Semmes OJ Schellhammer PF Yasui Y Feng Z Wright GL Jr. Adam BL, Qu Y. Serum protein fingerprinting coupled with a pattern-matching algorithm distinguishes prostate cancer from benign prostate hyperplasia and healthy men. *Cancer research*, 62(13):3609–3614, 2003.
- [2] Paweletz CP Ardekani A Hackett PS Hitt BA Velasco A Trucco C Wiegand L Wood K Simone CB Levine PJ Linehan WM Emmert-Buck MR Steinberg SM Kohn EC Liotta LA Petricoin EF 3rd, Ornstein DK. Serum proteomic patterns for detection of prostate cancer. https://home.ccr.cancer.gov/ncifdaproteomics/pdf/JNCI_prostate_paper.pdf, 2002.
- [3] Robert Tibshirani, Michael Saunders, Saharon Rosset, Ji Zhu, and Keith Knight. Sparsity and smoothness via the fused lasso. *Journal of the Royal Statistical Society: Series B*, 67(1):91–108, 2005.